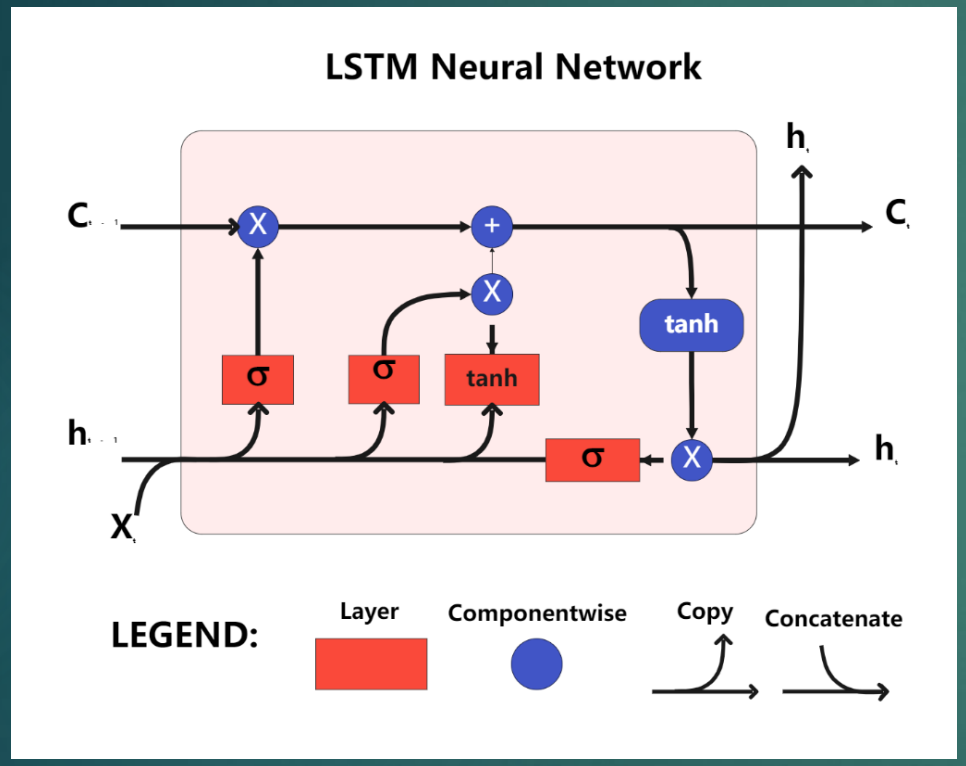
Introduction to LSTMs

EDINBURGH AI



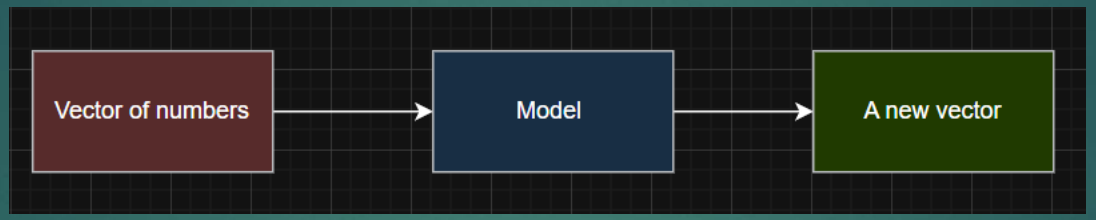
What we'll be looking at today

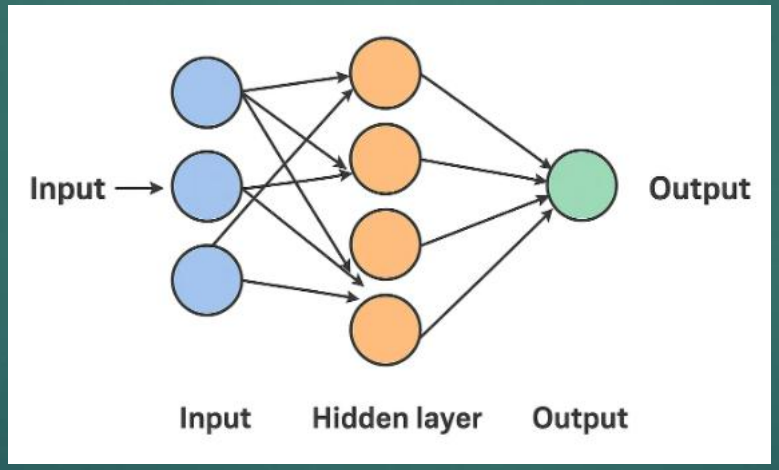




Recap:
What is a Neural Network?

Neural Network overview

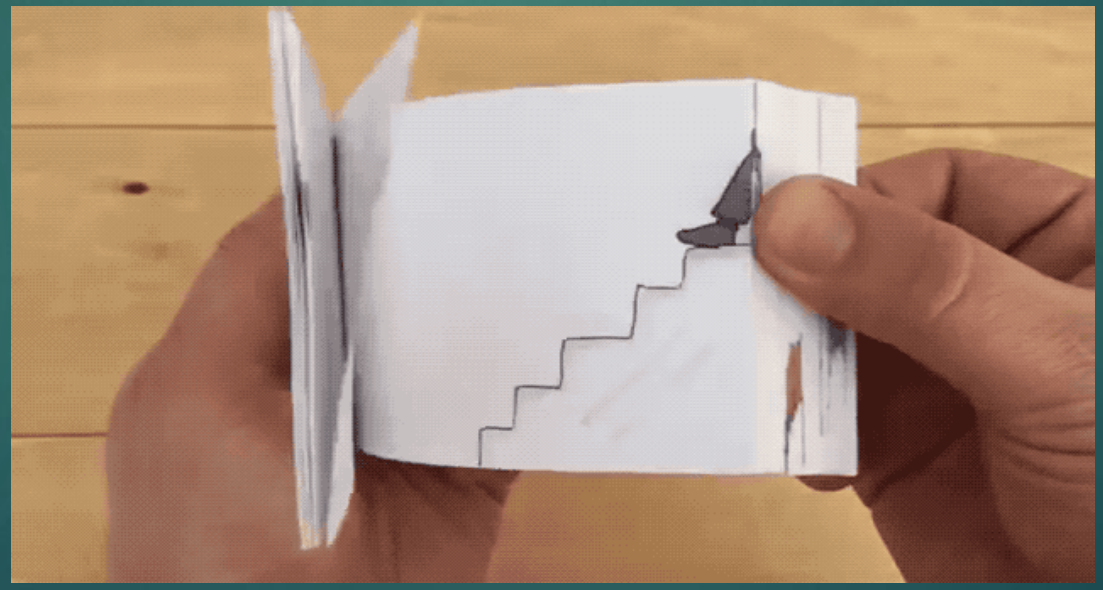




If Neural Networks expect lists of numbers as their input, how can we give them a video?

What really are videos?

- Videos are just sequences of lots and lots of images shown in the order they were taken. Like scrolling through a slide show really fast.
- We call each of these images a “frame”



But how does a computer even store a picture?

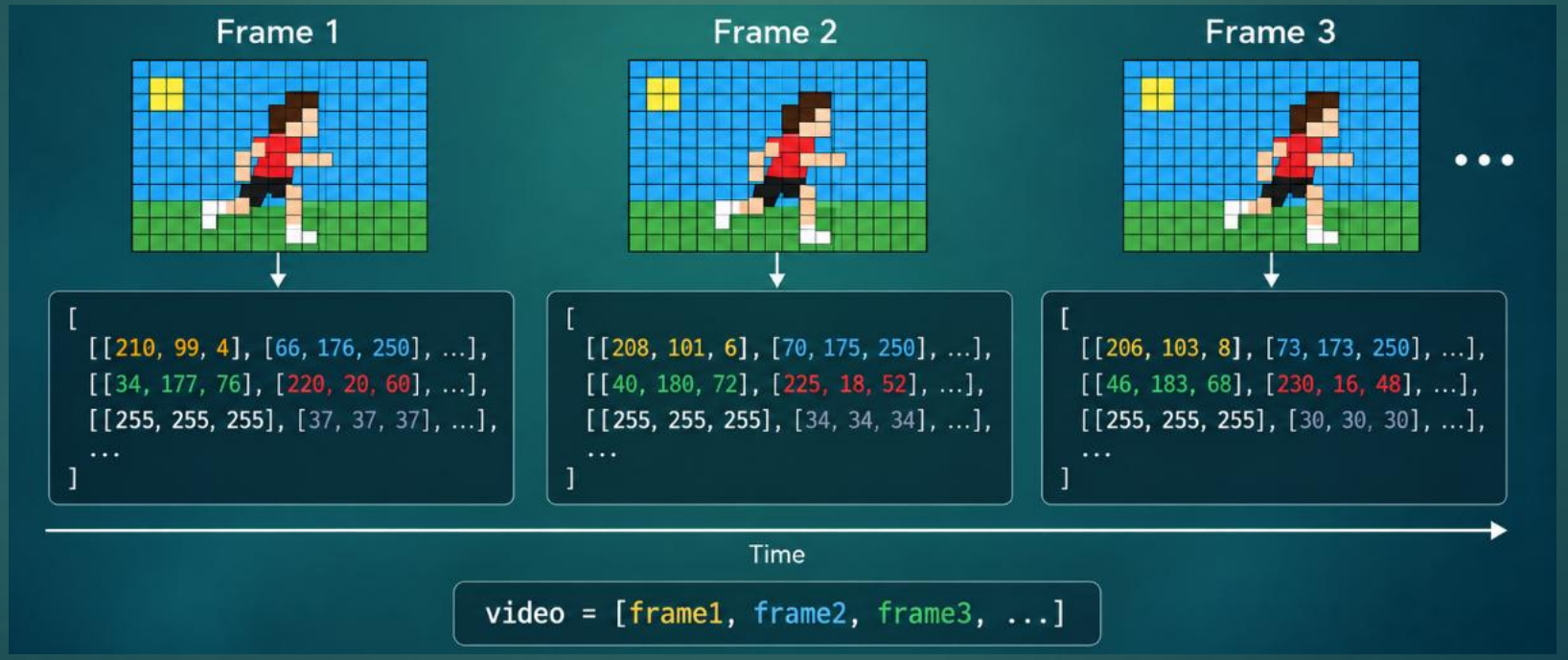
There are a few different methods you can use, but the easiest to get your head around is probably Bitmap.

When using Bitmap format, at its core, an image on a computer is just a matrix of pixels, and each pixel has a red, a green, and a blue value that together make a colour.

210, 99, 4	63, 22, 180	255, 12, 12
190, 45, 11	5, 150, 88	112, 44, 90
200, 91, 104	18, 18, 18	156, 32, 200

Bringing this together

This essentially means that a video is just a really long and fast slide show of all these bitmap colour grids.



One issue for sports analysis

When we're watching a sport, e.g. soccer, a lot of the time we don't actually care what is going on in the video apart from the players.

e.g. If something is happening in the background of the video, maybe a fan cheering, it usually doesn't really change our understanding of where the ball is on the pitch and what each player is doing

Solving this problem

What we can do is use another AI model that extracts the key coordinates of each player in every frame and returns a new sequence of lists of numbers that represents what we actually care about in the video.

The model often used for this is called YOLO, and usually it looks out for 17 points on each player and returns a list of numbers for each frame containing the (x,y) coordinate of each of those keypoints.

The 17 keypoints includes things such as:

- The person's left hip
- The person's right hip
- The person's right shoulder
- Etc.

Now it's time to design our Neural Network

Building a Neural Network that takes in a sequence of numbers

In previous workshops, such as the one on Linear Regression (intro to machine learning), the neural networks that we built only took in a single list of numbers, but here we have a **sequence** of lists of numbers.

The key thing we need to keep in mind is that because our frames are ordered chronologically, we can learn things as the sequence progresses.

e.g. We look at the first frame in our video, memorize something about it, then look at the next, and think how that changes our understanding of all the frames we've seen up until now.
Maybe the people in the frame moved?

Recurrent Neural Networks (RNNs)

Some terminology: “sequence”

The input for an RNN is a sequence of vectors

By a sequence we just many a list of data points in **chronological** order, taken at regular intervals.

e.g.

- Tracking a stock price for 8 hours and recording its price after every hour would give the sequence [hour1, hour2, …, hour 8]

For use in RNNs, if we change the order of the sequence

e.g. [hour3, hour 4, hour 7, hour2, …] It will break our RNN logic.

Recurrent Neural Networks (RNN)

Recurrent Neural Networks process data **sequentially –** instead of processing our input sequence in parallel (like a transformer does), they process one vector in our sequence at a time.

They effectively keep a memory of what the vectors they have already seen in the input sequence have told them.

e.g. Imagine a sequence of length 3

An RNN would follow the process:

1. Process vector 1 (The first vector) in the sequence and draw some conclusions from it and track of what conclusions it made from it.
2. Process vector 2 in the sequence, **then** use it to **update** what its current conclusion about the input sequence is – i.e. it updates its memory.
3. Process vector 3 in the sequence, then use it to update its understanding **again**

How do we represent this memory

To represent the RNNs memory we use what's called a hidden state vector, denoted h_t

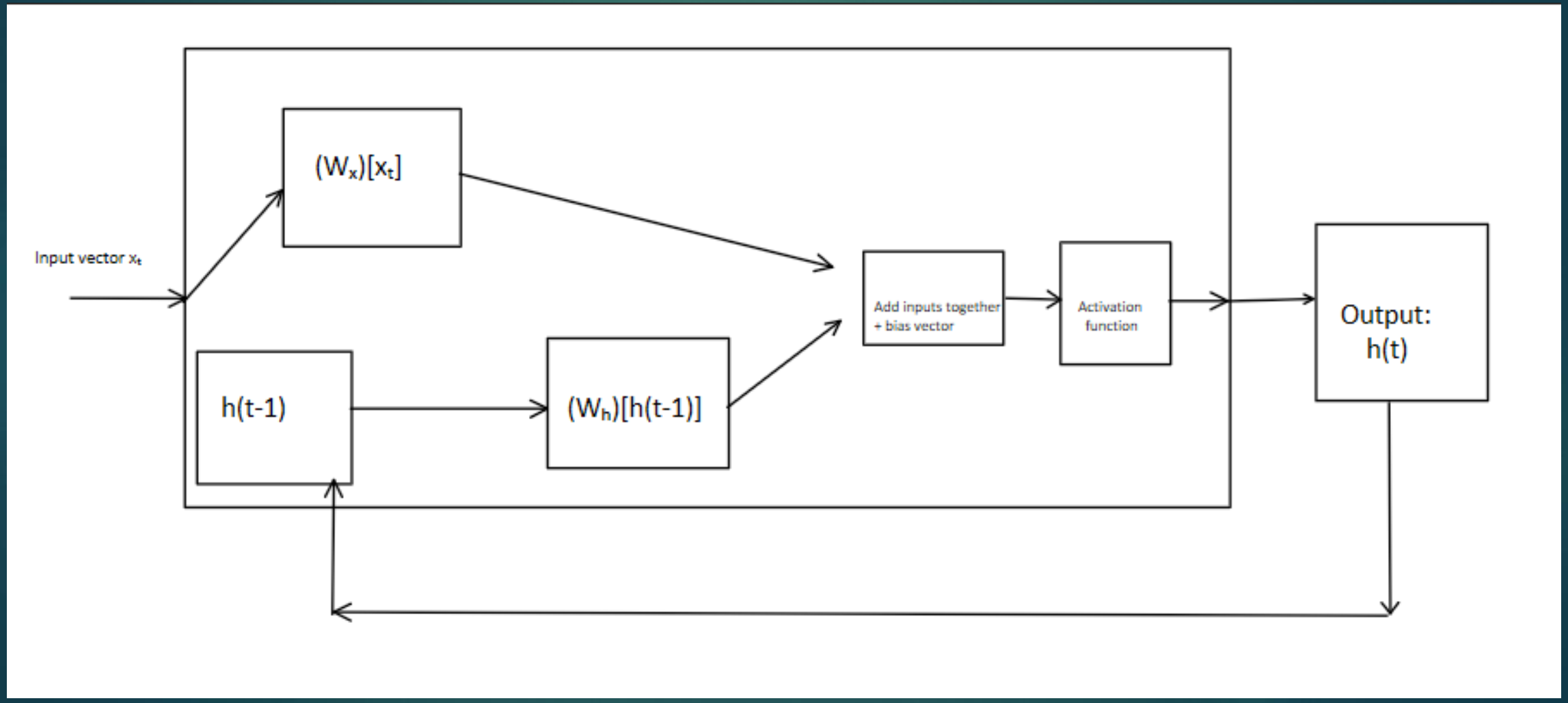
For each input vector:

1. We apply our standard linear layer logic (weighted linear transformation) to it
2. Apply another weighted linear transformation to our hidden state vector from the previous time step
3. Add the resulting vectors from step 1 and step 2 together
4. Add a bias vector to the vector produced from step 3
5. Apply an activation function to the vector produced in step 4 (commonly tanh is used as it's centered at 0 which can help with training)

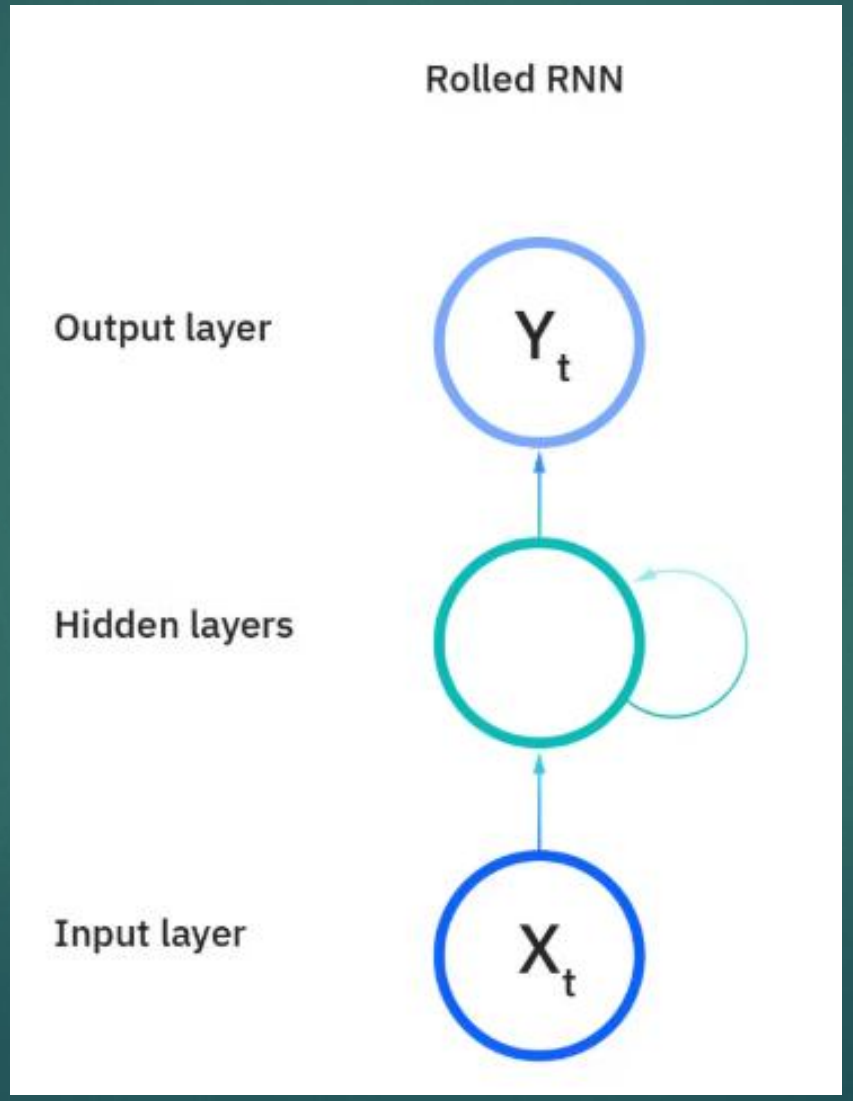
The vector you have after step 5 is the hidden state vector for that timestep, h_t.

You then save this hidden state vector for use in processing the next vector in the sequence.

Flow diagram of RNN



Traditional RNN diagram



What is the output of an RNN

The final hidden state vector for a sequence is the RNNs understanding of the sequence.

With this vector you have a couple of options:

- Pass it into another RNN (known as another layer) for further processing
- Pass it through a linear layer in order to get a vector that is in the shape of what you're actually trying to predict

We'll see both of these in the notebook later!

Training RNNs – Backpropagation through time

It's a very similar idea to how we've trained models before but we need to account for the fact that part of what causes the final output vector is the values in the hidden states part way through the processing the sequence.

If the RNN fails to predict the stock price at Hour 9 correctly, the blame (gradient) must flow:

1. From the **Loss** at the end.
2. Back to the **Hidden State at Hour 8**...
3. Then back through the temporal link to **Hour 7**, **Hour 6**, and all the way to **Hour 1**.

For those interested in the Maths

(Where E represents error aka “loss” or “cost”)

To find how the error (E) changes with respect to a weight (W), we sum the contributions from each time step t:

$$\frac{\partial E}{\partial W} = \sum_{t=1}^{T} \frac{\partial E_t}{\partial W}$$

Because each hidden state depends on the previous one, the gradient at time t involves a long product of derivatives:

$$\frac{\partial E_t}{\partial W} = \sum_{k=1}^{t} \frac{\partial E_t}{\partial y_t} \frac{\partial y_t}{\partial h_t} \left(\prod_{i=k+1}^{t} \frac{\partial h_i}{\partial h_{i-1}} \right) \frac{\partial h_k}{\partial W}$$

The main issue with RNNs

RNNs really struggle with long sequences. in practical application, standard RNNs struggle significantly once sequences exceed **10 to 30 time steps**.

Why do they struggle with long sequences?

The issue is actually caused by backpropagation through time.

Because you are multiplying many derivatives together (the product ∏ in the mathematical formula), if those numbers are small (less than 1), the gradient shrinks to almost zero by the time it reaches Hour 1.

How do we solve this issue?

Long Short-Term Memory (LSTM) Networks

Long Short-Term Memory (LSTM) Networks

The main idea of LSTM networks is to reduce the amount of times that the memory is multiplied by the derivative of the loss function.

They do this by introducing another vector in addition to the hidden state vector. This vector is called the Cell state vector, C_t

They also have to introduce some other operations to make it so the cell state doesn't vanish:

- Candidate Cell state ($\tilde{C}_t$)
- Forget Gate (f_t)
- Output Gate

TL;DR: Main components of an LSTM

- **Hidden state (h_t):** short term memory
- **Cell state (C_t):** long term memory
- **Candidate Cell state ($\tilde{C}_t$):** Conclusions we may want to add to long term memory
- **Input Gate:** The input gate's primary purpose is to regulate the flow of new information into the cell's long-term memory.
- **Forget Gate (f_t):** decides what information from the cell's long-term memory should be discarded or forgotten.
- **Output Gate:** decides what information in the long-term memory is most relevant at a given moment in time and then expose that information as the cell's output.

What do we mean by gates?

The idea of the gates is to restrict what information certain parts of the network see, to allow those weights (at those parts in the network) to focus on more important relationships.

- LSTMs have 3 gates: An input gate, a forget gate and an output gate.
- Each gate is a bit like a MLP Layer in the sense that it has: a weight matrix multiplying an input vector, a bias vector and an activation function (usually sigmoid).

LSTM Stage 1 – Computing the gates and candidate cell state

The Input Gate

The input gate's primary purpose is to regulate the flow of new information into the cell's long-term memory.

The inputs to the input gate are:

- The current input vector (x_t)
- The previous hidden state vector (h_{t-1})

These inputs are then concatenated to make a new vector:

- $[x_t, h_{t-1}]$

Once you have this concatenated vector you put it through a weighted linear transformation + activation function

Input gate computation and usage

The full equation for the input gate is:

$$i_t = \sigma(W_i \cdot [x_t, h_{t-1}] + b_i)$$

This means that the input gate is actually a vector.

Usage:

- We'll later see this input gate vector used in updating the cell state vector

The Forget Gate

The forget gate's primary purpose is to decide what information from the cell's long-term memory should be discarded or forgotten. It's the mechanism that prevents the cell state from being filled with old, irrelevant information.

The inputs to the forget gate are the same as the input gate's:

- The current input vector (x_t)
- The previous hidden state vector (h_{t-1})

These inputs are then concatenated to make a new vector:

- $[x_t, h_{t-1}]$

Once you have this concatenated vector you put it through a weighted linear **(with different weights to the one used for the input gate!)** transformation + activation function

Forget Gate Computation and usage

The full equation for the forget gate is:

$$f_t = \sigma(W_f \cdot [x_t, h_{t-1}] + b_f)$$

This means that the forget gate is also actually just a vector.

Usage:

- We'll later see this forget gate vector used in updating the cell state vector

Forget gate vs input gate

- The input gate acts on new information given in the current data point (token), deciding what to add to the long term memory.

- The forget gate acts on the information **already in the system** before the current data point and decides what information in the long term memory to forget, given the contents of the new data point.

TL;DR:

- The input gate's output vector (i_t) filters the candidate cell state vector ($\tilde{C}_t$).
- The forget gate's output vector (f_t) filters the previous cell state vector (C_{t-1}).

The Output Gate

The output gate's job is to decide what information in the long-term memory is most relevant at a given moment in time and then expose that information as the cell's output.

Once again, for computing the output gate we concatenate the input vector with the previous hidden state vector.

- $[x_t, h_{t-1}]$

Same as the other gates, we put this concatenated vector through a weighted linear transformation and then an activation function.

Output Gate computation and usage

The full equation for the output gate is:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

Once again, the output gate is actually just a vector.

Usage:

- ***The output gate vector created is used to compute the new hidden state vector***

Candidate Cell State

The candidate cell state is basically what we did in our RNN. It gives us all of the information that the short-term memory has to offer, so that we can then prune it.

It's simply just:

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Why tanh?

Why tanh for the candidate cell state activation function?

Sigmoid squashes numbers into a range between **0 and 1**.

This is perfect for a gate because it acts like a percentage of flow:

- Multiply by 0: The gate is fully closed (forget this).
- Multiply by 1: The gate is fully open (remember all of this).

However, The candidate cell state isn't a gate; it is the actual **new information** the network wants to write into the long-term memory.

tanh squashes numbers into a range between -1 and 1. This is crucial because it allows the network to add or subtract information from the cell state.

- A positive value means: "Increase the strength of this memory.“
- A negative value means: "Decrease the strength of this memory.

LSTM Stage 2 – Computing new cell state

Computing new Cell state

As said before, we want to use the information learned/blocked out by the gates in order to update in network's long-term memory (its cell state).

We create the new cell state vector by:

- Choosing what to forget from the current long-term memory
- Deciding what relationships from the new data point are relevant and worth putting into long term memory.

This is why we needed the input gate and forget gate!

Having created the forget vector and the input vector in stage 1, we can use these to scale the values both our candidate cell state and previous cell state.

Candidate Cell state equation

Where:

- i = input gate vector
- f = forget gate vector
- $\tilde{C}_t$ = candidate cell state

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

LSTM Stage 3 – Computing new hidden state

Computing the new Hidden State

The final stage of a cell is to calculate the hidden state vector (h_t) for the current time step.

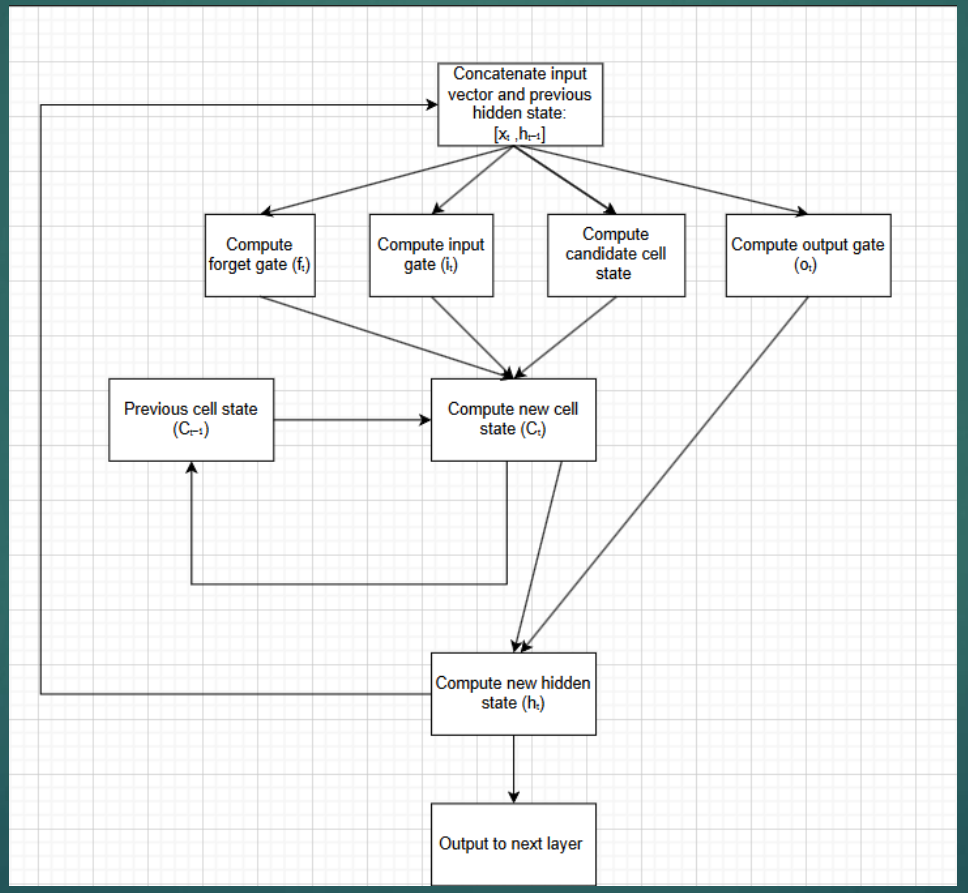
The hidden state vector also doubles as the output of the cell as it expresses what relationships were found to be important by that cell at the given time step.

Since the hidden state vector is about the **output** that timestep, we compute is using the **output gate**

Hidden state equation

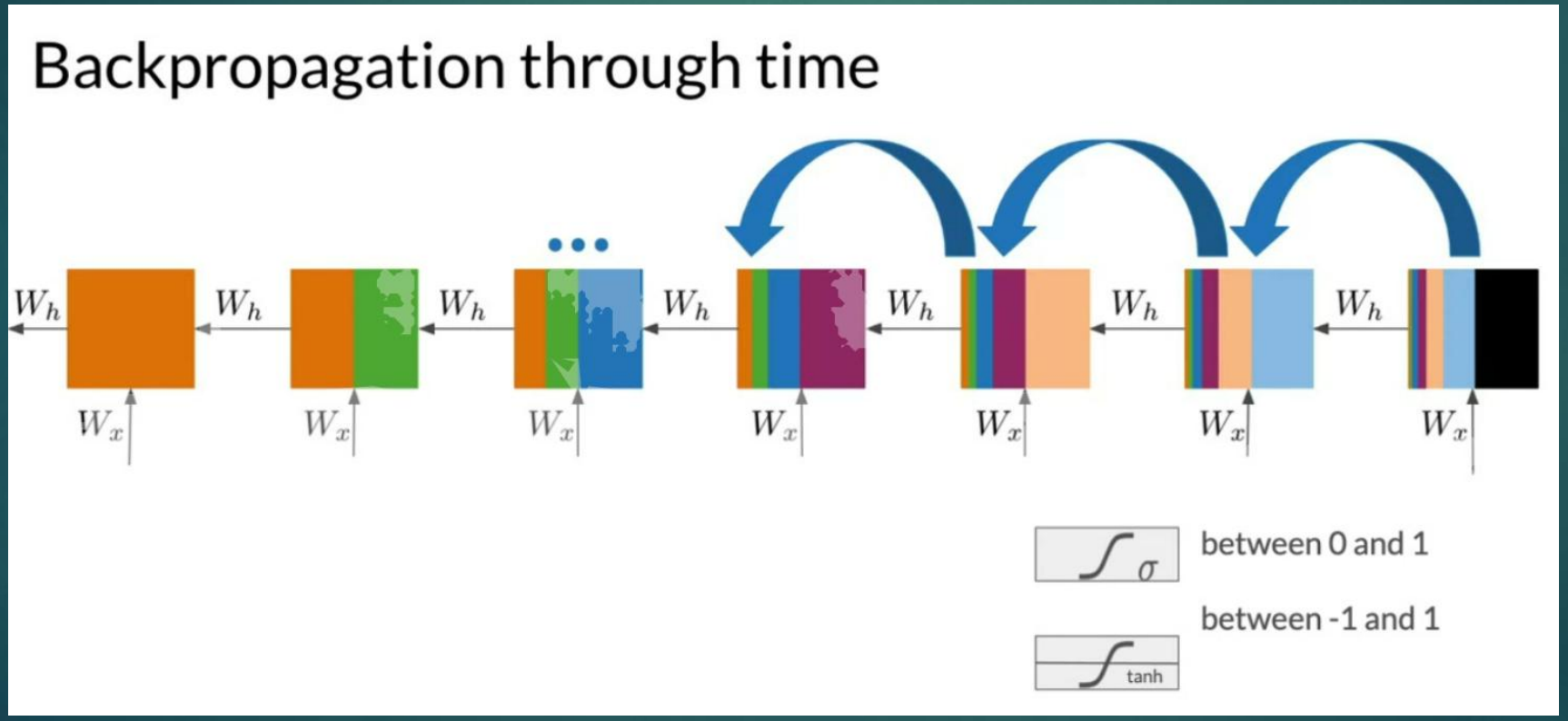
$$h_t = o_t \cdot \tanh(C_t)$$

LSTM Flow diagram



Training LSTMs

We once again use backpropagation through time



Why do LSTMs reduce the vanishing gradient issue?

The key idea is that when we compute our new cell state, we add components together instead of multiplying:

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

In calculus, the derivative of a sum distributes evenly. This means the error signal can flow backward through the addition operation without being mathematically squashed.

But are we not still multiplying in this equation?

This is why we have the forget and input gates!

Since the forget gate and input gate are computed using different weights:

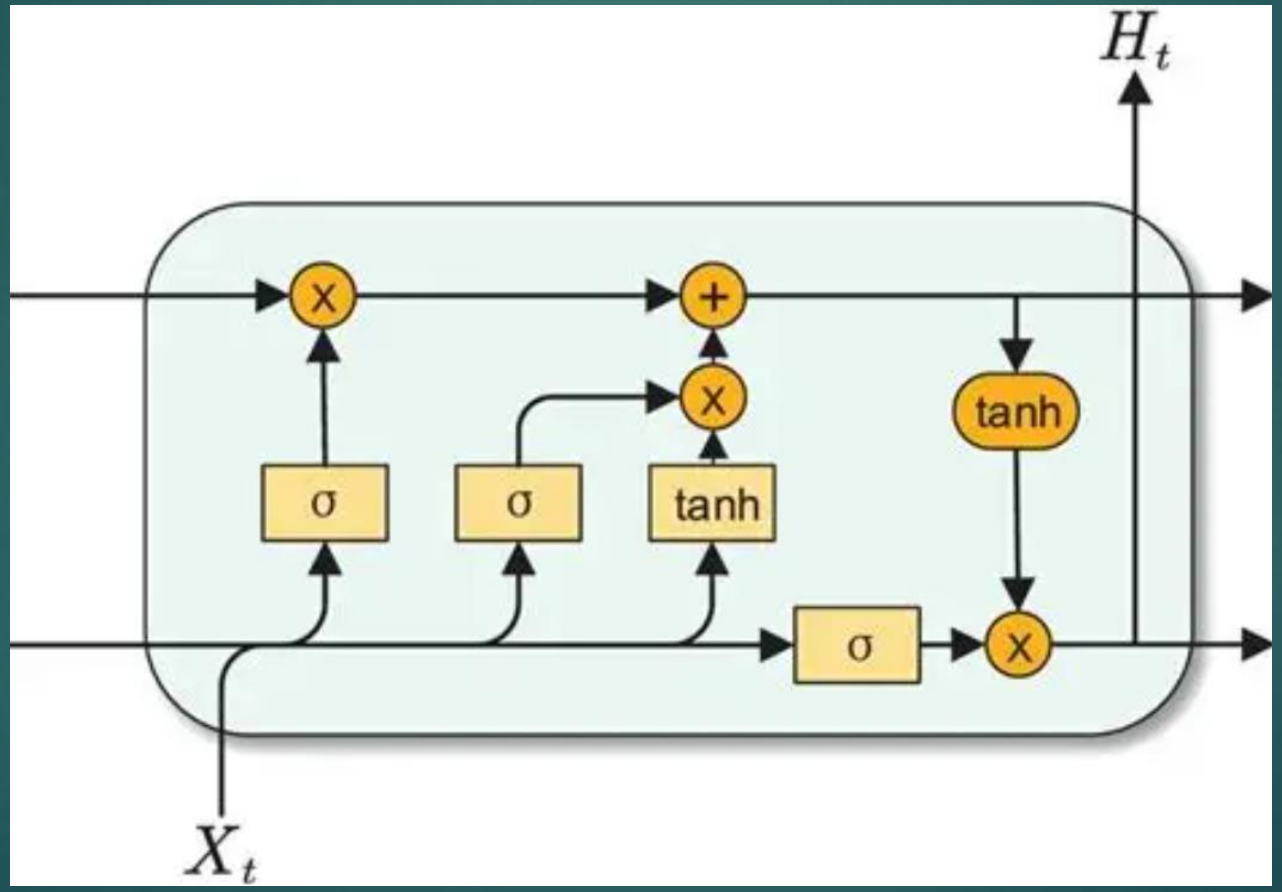
- If the information found previously is really important, the network will push the value of the forget gate closer to 1 so that it isn't lost
- If the information just found in the new time step is really important, the network will push the value of the input gate closer to 1 so that it isn't lost during backpropagation through time

Memory Highway

Another reason why LSTMs reduce the vanishing gradient issue is that the cell state isn't directly exposed to non-linear activation functions.

No Squashing on the Highway: The core long-term memory travels step-to-step without being forced through non-linear activation functions, avoiding the mathematical decay seen in standard RNN hidden states.

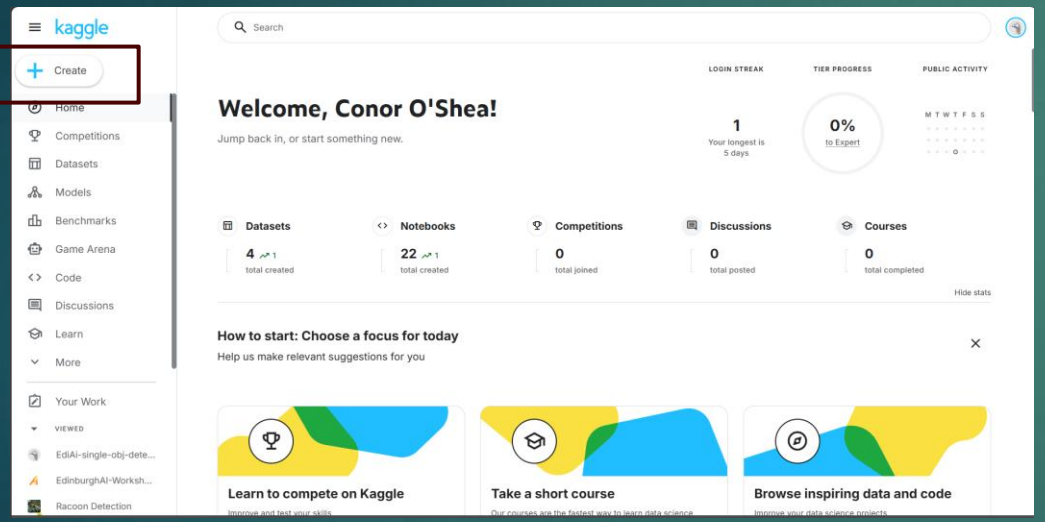
Traditional LSTM Diagram (Memory Highway)



Now it's your turn to implement these models!

Kaggle Setup

1. Go to: https://www.kaggle.com/
2. Create an account (if you don't have one already)
3. Click the create button in the top left and select "Import Notebook" from the dropdown menu.



5. Select "Link"
6. Enter the "unsolved" link posted on the workshops group on the WhatsApp. Links are also on the final slide of this deck.

Notebook URLs

Beginner:

https://github.com/EdinburghAI/workshops/blob/main/TheoreticalWorkshops/intro-to-rnn-lstm/BEGINNER-sports-vision-rnn-lstm.ipynb

Advanced:

- Notebook for completeion:

https://github.com/EdinburghAI/workshops/blob/main/TheoreticalWorkshops/intro-to-rnn-lstm/UNSOLVED-advanced-sports-vision-rnn-lstm.ipynb

- Solutions

https://github.com/EdinburghAI/workshops/blob/main/TheoreticalWorkshops/intro-to-rnn-lstm/SOLVED-advanced-sports-vision-rnn-lstm.ipynb